

Beyond Decode Speed: Exact Token Admission and Real-Workspace Evaluation at 256K Context

Chengyi Xu and KLIK team
KLIK Research

September 2026 — experimental systems report

Abstract

A local agent can stream quickly and still fail to finish useful work. We examine this distinction on an Apple M4 Max with 64GiB unified memory using real Pi sessions in isolated copies of a private software repository. Normal and ablated Ornith-1.5-35B-A3B Q4 models process more than 256,000 actual input tokens with ordinary repository tools and installed extensions. In an unmodified-client full-extension series, the normal model finishes only one of three runs; two continuations are restricted to 25 and one output tokens by an approximate context estimator and a fixed 4,096-token reserve. A local, exact-token admission hook restores available output capacity without truncation, compaction, or context-window inflation. Exact budgeting alone improves observed completion to three of three but does not fix instruction fidelity. A separately labelled condition combining native JSON-schema decoding and explicit verbatim-command instructions produces three of three bounded audit passes for each model. Visible streaming is 45.82–96.58 tokens/s for the normal model and 44.34–48.13 for the ablated model in that condition. Full final-turn rates are lower, including values below 20 tokens/s; cold prefill remains approximately 23–34 minutes in measured conditions. Broader repository reviews expose factual errors. We release sanitized measurements and measurement code, not a claim of universal intelligence or an end-to-end minimum-speed guarantee.

1 Motivation and scope

The practical objective is not an empty-session speed record. It is a local model that reads implementation files, executes useful checks, retains a long working context, and produces a correct answer through the actual agent client. Earlier selected-prompt “champion” results were insufficient evidence of this objective. In particular, historical 809-token/s LocalFlash results are retained as an unvalidated anomaly, not accepted performance. We corrected the public repository description that still advertised that number.

This report separates three questions: can a runtime accommodate the input, can an agent complete the workflow, and is its answer correct? A positive answer to the first does not establish either of the others. We also distinguish installed deployment mechanics from model qualification. Both endpoints and an image service can start correctly while a text model remains unreliable on particular tasks.

2 Environment and candidate selection

The workstation has an Apple M4 Max with 40 GPU cores and 64GiB unified memory, running macOS 27 build 26A428. It is a working desktop rather than a quiescent benchmark appliance. Host observations included approximately 40 GB of swap usage and hundreds of runnable processes.

Unrelated applications were not terminated. These observations are confounders, not proof that contention caused any specific error.

The measured text runtime is Splash at commit `c64a578fcfaa059023a2ca57230d4aeadd134b73b`, with local patches accepting Q4_K GDN alpha/beta tensors and an opt-in 512-row prefill cap. The native watchdog is unchanged. The final comparison preset uses 28 GiB Metal allocation and an 8 GiB SSD cache tier. Only one large engine runs at a time. A matched Qwen-family DFlash2 draft is shared as an architecture-compatible draft, not substituted for either target model.

The normal target is `ornith-ai/Ornith-1.5-35B-A3B-GGUF`.

Its pinned revision is `12393612fd4f730ff5aadcd23e9b8f9648aa49ceb`.

The separate ablated target is

`PocketAiHub/Ornith-1.5-35B-A3B-Abliterated-GGUF`.

Its pinned revision is `78c2971382a49b442ce044fd4e17169217b0b36c`. Both are Q4_K_M files with pinned size and SHA-256 verification. “Abliterated” describes the checkpoint provenance; no universal non-refusal behavior or safety characteristic is established by the coding tests.

We also measured an Empero 35B-A3B distillation in the same native preset and repository-review task. It was not selected: its answer contained major call-path and deadline errors despite usable streaming speed. Its card itself warns of possible long-context degradation from 8,192-token training examples. Names suggesting a frontier teacher are not intelligence evidence.

3 Research coverage and non-transferable claims

DFlash2, oMLX, mlx-dspark, Dynamic V3, Rapid-MLX, mlx-serve, Prism/Bonsai, Splash and its LM Studio integration, and FreeToken were examined through source trees or captured primary materials [1–10]. Their relevance differs. Speculative decoding depends on draft acceptance and verification cost; quantization changes memory and quality, not simply speed. Hybrid models require recurrent-state as well as KV-cache reuse. A packaging layer does not constitute independent acceleration.

FreeToken’s implementation targets a different hardware/software environment, so its reported acceleration is not a Metal measurement. Flash-Next-scale checkpoint sizes and resident-memory claims were screened against the 64 GiB host and long-context headroom rather than downloaded indiscriminately. Earlier Bonsai and dense-Qwen configurations failed measured latency/throughput or completion gates; that rejects those configurations, not all possible engines for those families. The accompanying research ledger records pinned sources and remaining experimental gaps. This work does not claim an exhaustive global model ranking.

4 Tasks and evidence boundaries

Repository review (A). The agent traces an OpenAI-compatible structured request through the public facade, retries, deadlines, overrides and tests, then writes a 400–650-word report. This is a freeform intelligence check. Actual tool traces and cited source were inspected; proposed test commands were checked separately.

Repository repair (B, earlier evidence). A deliberately injected retry-cap defect in an isolated checkout must be repaired. This is not represented as a newly discovered production bug. Separate gates and full unit suites passed in earlier normal and ablated runs; those results do not automatically transfer to every later configuration.

Deep audit (D). A reference attachment contains real private source snapshots and three position-separated receipts. The agent reads the actual retry implementation and uses the checkout’s Python

environment to execute three retry cases plus HTTP-status checks. It must recover the receipts from supplied context, not reread the attachment, and report the actual successful command. The expected delay sequences are $[1, 2]$, $[2, 2, 2, 2]$, and $[]$; status 429 is retryable and 401 is not. These are evaluation targets, not injected answer values in generation constraints.

Normal extension discovery exposes 38 tools. The original controlled attachment plus full tool schemas exceeded the model limit at 267,827 tokens. We therefore created an explicitly separate full-extension reference variant, removing 11 complete late source sections while preserving all receipts and the task. It contains 155 source sections and produces 256,150 actual initial input tokens. The controlled and full-extension datasets are not pooled as identical conditions.

Verbatim condition (D-V). D-V adds an explicit instruction to copy the successful command character-for-character, preserving newlines through JSON escaping. Initial input is 256,222 tokens. Native JSON schema specifies output structure but supplies no expected receipt values, numeric answers, or command enums. No response is repaired after generation. The original instruction condition’s failures remain failures.

5 Measurement definitions

Let N_v be the final visible text retokenized by the target tokenizer. Let t_v be its first nonempty visible delta, t_e the final message end, t_s the assistant-stream start, and t_c the agent turn start. We report

$$R_{\text{stream}} = \frac{N_v}{t_e - t_v}, \quad R_{\text{cycle}} = \frac{N_v}{t_e - t_c}.$$

The first excludes preceding reasoning and prefill. The second includes turn-level preparation, exact-token preflight, reasoning and waiting. The older assistant-request interval $t_e - t_s$ is retained but is not identical to the full cycle because the provider hook can run before assistant-stream creation. Whole-task elapsed includes all turns and tools. Whole-task completion throughput includes reasoning and tool syntax unless explicitly stated otherwise; it is not visible-output throughput.

Tool parsers may buffer tool-call deltas, so tool-only client bursts are not backend decode evidence. A one-token cutoff once yielded a numerical artifact near 197 tokens/s over about 5 ms. The harness now reports such sub-0.1-second decode windows as unknown. Characters and SSE chunks are never counted as tokens. Timeout, error, length cutoff and missing usage are not successes.

6 Exact local admission

Pi’s observed output cap is

$$B_{\text{Pi}} = \min(B_{\text{model}}, \max(1, C - \hat{N} - 4096)),$$

where $C = 262144$ and $\hat{N}$ is an input estimate. Replaying captured messages yields $\hat{N} = 258023$ and 258134 in two failed normal runs, reproducing caps of 25 and one. Actual input counts are 258088 and 258228, leaving thousands of physical model-context positions even though the client reserves nearly all of them.

Our explicitly local request hook uses the same backend’s template renderer and tokenizer, then sets

$$B_{\text{exact}} = \min(4096, C - N_{\text{exact}} - 64).$$

If no positive budget remains, it fails visibly. It does not compact, truncate, enlarge C , or change tools/messages. The 64-token margin is a declared engineering choice, not a model-context extension.

The endpoint identity is restricted to the two local providers; redirects and cloud endpoints are rejected. Counting failure aborts the request. This is important because the client catches transformation-hook exceptions: throwing alone would leave the original request intact.

Every recorded preflight count in the exact-budget series matched the corresponding backend input usage. In one actual normal continuation, an old cap of one became 3,773 at 258,307 input tokens; the model emitted a complete answer. Three-turn preflight overhead was not assumed free: logged CPU counting time and full-cycle timing are included. Global installation is scoped to local profiles, with an explicit opt-out, and does not change cloud defaults.

7 Results

Condition	Completed	Bounded quality	Interpretation
Normal, unmodified full extensions	1/3	0/3	Two client length cutoffs
Abliterated, unmodified full extensions	3/3	0/3	Formatting/command failures
Normal, exact budget only	3/3	0/3	Completion is not correctness
Normal, exact budget + schema	1/1	0/1	JSON fixed; command abbreviated
Normal, exact budget + schema + D-V	3/3	3/3	Bounded audit only
Abliterated, same D-V method	3/3	3/3	Independent target evidence

Table 1: Small descriptive series, not a randomized causal estimate. Sampling and host state vary; instruction conditions are explicitly separated.

Target	Run	Max input	Wall (s)	R_{stream}	R_{cycle}
Normal	D-V 1	259363	124.57	56.77	16.72
Normal	D-V 2	258971	56.74	96.58	40.95
Normal	D-V 3	259332	86.38	45.82	26.31
Abliterated	D-V 1	257133	486.80	44.34	16.34
Abliterated	D-V 2	257037	34.45	45.91	23.70
Abliterated	D-V 3	258408	86.29	48.13	19.47

Table 2: Actual Pi workspace outputs; token rates are tokens/s. Normal D-V runs reuse a prepared prefix. Abliterated D-V1 retains 212,992 cached tokens from an interrupted attempt and is neither fully cold nor fully warm. The interrupted attempt is preserved as an infrastructure interruption, not silently discarded as a model success.

The full-extension cold normal baseline takes 1,765.78 seconds and the cold abliterated baseline 1,810.99 seconds. Another normal controlled production run takes 2,024.06 seconds. Exact-budget normal cold inference takes 1,462.49 seconds. These values do not establish a prefill speedup from admission control: host load, state and runs differ. Cold latency remains a major practical limitation.

Freeform review is less reliable than the bounded execution audit. Normal Ornith completes in 260.78 seconds, with 20 tools and no tool errors; its 562-word report has broadly supported core findings but overstates structured defaults and omits a classifier precedence detail. Its visible streaming is 43.55 tokens/s, but its full final-cycle rate is 13.40. The abliterated review completes in 338.93 seconds and misidentifies part of the factory path and repair behavior. Empero completes in 345.77 seconds with 27 tools and a 1,077-word report; it invents sync delegation and incorrect shared-deadline arithmetic. It is not selected despite 39.01 visible-stream tokens/s. No claim of flawless reasoning follows from the deep audit passes.

8 Deployment and reproducibility

The two text endpoints and a separate image service are exposed through an on-demand Desktop controller and registered in CC Switch. Text providers remain distinct from the image MCP tool. Startup, identity/readiness, exclusive switching and shutdown were exercised. Only one large runtime is admitted. Registration is not an assertion that every registered client has independently passed a deep test.

The public artifact contains sanitized numerical evidence, measurement code, tests, model revisions and method descriptions. Private source snapshots and raw source-bearing transcripts are intentionally withheld. This limits independent reproduction of exact task inputs. We provide the generic task specification and equations so others can reproduce the method with their own repository, but do not present such substitutions as byte-identical replications.

9 Limitations and conclusion

Samples are small, stochastic, and collected under variable workstation contention. Quantization parity is bounded by implemented preparation tests, not a full high-precision model equivalence claim. No globally optimal model has been established. Stronger wording and grammar constraints change the method condition; the resulting passes must not replace the original failures in tables.

The central result is narrower and more useful than a headline speed record: exact local admission can prevent a real client-induced cutoff, while native output constraints and explicit reporting instructions can improve a bounded tool audit. Fast visible streaming does not guarantee a fast complete request, fast cold start, or correct repository reasoning. A universal 20-token/s end-to-end guarantee is not supported.

Disclosure and status

This is an AI-assisted experimental systems report. Automated tools assisted implementation, measurements, source comparison and drafting. No independent human manuscript review, peer acceptance, funding declaration or conflict attestation is asserted. Publication of this artifact is hosting, not journal acceptance. Author naming follows the project owner’s requested attribution. Model and runtime licenses remain separate; weights are not redistributed.

References

- [1] Z-Lab. DFlash. <https://github.com/z-lab/dflash>.
- [2] oMLX. <https://github.com/jundot/omlx>.
- [3] mlx-dspark. <https://github.com/ARahim3/mlx-dspark>.
- [4] Unsloth. Dynamic quantization documentation. <https://docs.unsloth.ai/>.
- [5] Rapid-MLX. <https://github.com/raullenchai/Rapid-MLX>.
- [6] mlx-serve. <https://github.com/ddalcu/mlx-serve>.
- [7] Prism ML model collection. <https://huggingface.co/prism-ml>.
- [8] Inco. Splash. <https://inco.ai/blog/splash>.

- [9] LM Studio. Splash engine. <https://lmstudio.ai/blog/splash-engine>.
- [10] FlashML. FreeToken. <https://github.com/FlashML-org/FreeToken>.
- [11] Ornith model card. <https://huggingface.co/ornith-ai/Ornith-1.5-35B-A3B>.
- [12] Empero model card. <https://huggingface.co/empero-ai/Qwen3.8-35B-A3B-Distill>.